

Application Architecture

Questa lezione introduce l'**architettura a livelli (layered architecture)** che sarà usata in tutta l'applicazione.

Definizione — Architettura a livelli: è un pattern in cui il codice viene diviso in strati con responsabilità separate, dove ogni strato parla solo con quello immediatamente sotto di sé. Questo si chiama *separation of concerns* (separazione delle responsabilità).

I tre livelli sono:

- **Controller layer (livello REST):** contiene gli endpoint REST. È il punto di ingresso delle richieste HTTP. La sua **unica** responsabilità è ricevere la richiesta, passarla al livello sotto, e restituire la risposta. Non deve contenere logica di business.
- **Service layer (livello di servizio):** qui va la **logica di business** e le **validazioni**.
- **Repository layer (livello modello/database):** si occupa di parlare con il database usando JPA e Spring Data.

Lo stack tecnologico scelto:

- **Spring Boot 3** come framework
- **Tomcat** come web server (arriva automaticamente con Spring Web)
- **JPA** per la persistenza
- **H2** come database *in-memory* (in memoria)
- **Lombok** come strumento di sviluppo
- **Postman** come client per testare

Definizione — Database in-memory (H2): un database che vive nella RAM. È velocissimo, ma quando chiudi l'applicazione tutti i dati spariscono. Perfetto per demo e test.

Definizione — Design RESTful e risorse: in REST ogni URI rappresenta una *risorsa*. Il docente distingue due tipi:

- **Collection resource** (risorsa di raccolta), es. `/cars` → rappresenta *tutte* le auto
- **Individual resource** (risorsa individuale), es. `/cars/{regNo}` → rappresenta *una* auto specifica

I verbi HTTP hanno significati precisi per ogni tipo di risorsa. L'esempio del corso:

- `GET /cars` → recupera tutta la collezione
- `POST /cars` → aggiunge una nuova auto **alla collezione** (POST va solo sulla collezione)
- `PUT /cars/{regNo}` → aggiorna una singola auto (PUT va solo sulla risorsa individuale)
- `DELETE /cars/{regNo}` → elimina una singola auto; `DELETE /cars` cancella tutte
- Un POST su una risorsa individuale → **non permesso** (405 Method Not Allowed)

Vengono anche introdotti i **request parameters** (parametri di richiesta), es. `/cars/car?brandName=Ford` per filtrare tutte le auto Ford, oppure `/cars/car?carType=electric` per tutte le elettriche. Il `?nome=valore` dopo l'URI è il parametro di richiesta.

Spring Boot Setup (start.spring.io)

Definizione — start.spring.io (Spring Initializr): un sito web che genera lo scheletro di un progetto Spring Boot pronto all'uso, così sei operativo in pochissimo tempo.

Le scelte fatte nella demo:

- Build tool: **Maven**
- Linguaggio: **Java 17**
- Packaging: **Jar**
- Group: `com.preowned`, Artifact: `cars`

Le **dipendenze** selezionate (il punto cruciale):

- **Spring Web** → porta con sé Apache Tomcat
- **H2 Database** → il database, supporta JDBC
- **Spring Data JPA** → si appoggia sopra a tutto per l'accesso ai dati
- **Lombok** → annotazioni extra per scrivere meno codice

Cliccando *Generate*, il sito scarica uno zip. Lo si decompone e lo si importa in **IntelliJ** (import da "existing sources", dal `pom.xml`).

Definizione — pom.xml: è il file di configurazione di Maven. Elenca la versione di Spring Boot, la versione di Java (17) e tutte le dipendenze. È generato automaticamente da Spring Initializr. Il progetto segue la struttura standard di Maven (`src/main/java`, `src/main/resources`).

L'applicazione generata di default parte già su Tomcat, porta **8080**.

H2 DataBase Configuration

Qui si configura come l'applicazione si collega al database H2. Ci sono due file chiave nella cartella `resources`:

File `application.yml` (formato YAML)

Definizione — YAML: un formato di configurazione che usa l'**indentazione con spazi** per esprimere la gerarchia. L'indentazione è significativa: se sbagli gli spazi, cambi il significato.

Nel file vengono configurati:

- La porta del server (8080)
- Il **datasource** H2: `URL jdbc:h2:mem:testdb`, driver class `org.h2.Driver`, username `sa`, password vuota
- La **console H2 abilitata** (`enabled: true`)
- Impostazioni **JPA/Hibernate**: `ddl-auto: update`, `show-sql: true`, dialetto `Hibernate`

File `schema.sql`

Questo file viene eseguito **automaticamente ad ogni avvio** dell'applicazione e crea la tabella. Poiché H2 è in-memory, i dati spariscono ad ogni chiusura, ma questo file ricrea sempre la tabella vuota così non devi farlo a mano.

Nella tabella auto:

- `id`: chiave primaria auto-incrementale generata dal database (unica per ogni record)
- campi stringa: `brand_name`, `model_name`, `region`, `car_type` (varchar di varie dimensioni)
- campi interi: anno, chilometri, prezzo

Attenzione (gotcha importante): `year` (anno) è una **parola riservata** in SQL. Nell'editor diventa blu ed è un segnale che causerà problemi. Va evitato o gestito.

Per verificare: si va nel browser su `localhost:8080/h2-console`, ci si connette e si vede la tabella vuota — esattamente come deve essere ad ogni nuovo avvio.

High-level Packages and ApplicationContext Overview

Questa lezione spiega **come Spring Boot scopre e gestisce i bean**.

Definizione — Spring Bean: un'istanza di una classe **gestita dal framework Spring** (non da te). Tu non fai `new`, ci pensa Spring a creare, configurare e fornire l'oggetto quando serve.

Definizione — @SpringBootApplication e component scan: la classe main è annotata con `@SpringBootApplication`. Al suo interno c'è un *component scan* che dice a Spring di analizzare tutti i package a partire da quello base (es. `com.preactor.cars`) e da tutti i package annidati sotto. Ogni classe marcata come componente diventa automaticamente un bean gestito nell'**ApplicationContext**.

Definizione — ApplicationContext (Spring container): il contenitore che ospita tutti i bean gestiti da Spring.

Punto importante: se metti una classe in un package **fuori** dal package base, non verrà rilevata come bean, anche se la annoti come componente. `@Component` è l'annotazione fondamentale per dire "voglio che questa classe sia un bean".

Esempio pratico mostrato: si prende l'**ApplicationContext**, si chiama `getBeanDefinitionNames()` (che restituisce un array), lo si mette in stream e si stampano tutti i nomi dei bean a video. Si vede una lunga lista, inclusa l'applicazione stessa e Tomcat avviato sulla 8080.

Controller Layer Overview

Panoramica del **livello controller**, il punto d'ingresso dell'app.

Definizione — @RestController: un'annotazione *stereotipo* che classifica una classe come controller REST. È uno "stereotipo" perché comunica sia agli sviluppatori sia al framework che questa classe ha quel ruolo specifico. Al suo interno è a sua volta un `@Component`, quindi diventa un bean.

Definizione — @RequestMapping: definisce il percorso base a cui il controller risponde (es. `/cars`) e cosa produce (`produces = JSON`).

Definizione — Dependency Injection (DI) e Inversion of Control (IoC): questi sono i concetti centrali di Spring.

- **Inversion of Control:** invece di creare tu stesso gli oggetti di cui hai bisogno, "cedi il controllo" al framework. Non fai `new`, lasci fare a Spring.
- **Dependency Injection:** il framework *inietta* (fornisce) le dipendenze di cui una classe ha bisogno.

Esempio concreto: la classe `CarController` **dipende** da un'implementazione del service (`ICarService`). Grazie all'annotazione Lombok `@AllArgsConstructor` (che genera il costruttore con tutti gli argomenti), Spring può iniettare automaticamente il service nel controller. Tu dichiari solo la variabile d'istanza; il framework la popola.

Il controller contiene tutte le *mapping*:

- `@GetMapping` → recuperare
- `@PostMapping` → aggiungere
- `@DeleteMapping`
- Mapping specifici come `/car?brandName=...` e `/car/{regNo}`
- PUT sulla collezione → restituisce "method not allowed"

Definizione — @ControllerAdvice e gestione globale delle eccezioni: il controller ha anche un **gestore di eccezioni globale** annotato con `@ControllerAdvice`. Tutte le eccezioni lanciate nell'applicazione vengono instradate qui, in un unico posto, permettendo di arricchire il messaggio di risposta al client. Questo evita di ripetere la gestione errori ovunque.

Service Layer Overview

Il **livello di servizio** contiene logica di business e validazione.

Definizione — Interfaccia + implementazione: il service è definito prima come **interfaccia** (`ICarService`) che elenca i metodi, poi come **classe di implementazione** (`CarServiceImpl`). Questo è un pattern classico: l'interfaccia dice *cosa* si fa, l'implementazione dice *come*.

L'implementazione è annotata con `@Service`, che è un tipo di `@Component`, quindi è un bean di Spring. (Paralelo: il controller usa `@RestController`, il service usa `@Service` — entrambi comunicano il ruolo del layer.)

Il service dipende dal `CarRepository` (iniettato via costruttore Lombok). Di nuovo IoC: il service non istanzia il repository, lo lascia fare al framework.

Metodi tipici: aggiungere auto, ottenere tutte, ottenere una, ottenere per marca, eliminare, aggiornare.

Definizione — DTO (Data Transfer Object): questo è un concetto chiave. Quando salvi un record nel database, il DB genera la chiave primaria (`id`). Ma **non vuoi restituire la chiave primaria al client**, perché indebolisce la sicurezza (rischio di **SQL injection**: se il client conosce la chiave primaria è più facile attaccare il DB). Il `CarDTO` ha tutte le proprietà dell'auto **tranne** l'`id`.

Definizione — Mapper: il `CarMapper` prende un'entità (una riga del database, con l'id) e la mappa in un **DTO** (senza id). In pratica copia tutte le proprietà tranne la chiave primaria. È il DTO ciò che viene restituito al client.

Repository Layer Overview

Il **livello repository**, quello che parla col database. Qui c'è una delle parti più "magiche" di Spring Data.

Definizione — JpaRepository e query derivate: il repository è un'interfaccia che estende `JpaRepository<Entity, Long>`. Passandogli l'entità (la tabella) e il tipo della chiave primaria (`Long`), ottieni **gratis** una marea di metodi già implementati (`save`, `findAll`, `deleteAll`, ecc.) — non devi scrivere il codice né l'SQL.

Puoi anche definire **query derivate** (derived queries): metodi personalizzati che JPA implementa automaticamente **basandosi sul nome del metodo**. Esempi:

- `findByRegional(...)` → JPA genera l'SQL per cercare in base alla proprietà `regional`. Restituisce un `Optional<Car>`.
- `findAllByBrandName(...)` → cerca per la proprietà `brandName`.

Regola cruciale: il nome del metodo deve corrispondere esattamente al nome della **proprietà nell'entità**. Se scrivi `findByRegional`, devi avere una proprietà `regional` nella classe entità.

Definizione — Entity (@Entity): la classe che rappresenta una tabella. Nel corso l'entità `Car` è annotata con:

- `@Entity` → è un'entità persistente
- `@Id` → definisce la chiave primaria
- `@GeneratedValue(strategy = IDENTITY)` → l'id è generato dalla colonna `identity` del database (auto-incremento: 1, 2, 3...)
- `@Column(name = "...")` → serve **solo quando** il nome della proprietà Java differisce dal nome della colonna. Esempio: la proprietà Java `brandName` (camelCase) mappa alla colonna `brand_name` (con underscore). Se il nome della proprietà coincide già con la colonna, `@Column` non serve.
- Lombok genera getter, setter, `toString`, costruttori.

Definizione — Optional: un contenitore che *può* avere un valore o essere vuoto.

`findByRegional` restituisce `Optional<Car>`: o contiene l'auto, o è vuoto (mai null). Si usa `isPresent()` per controllare in modo funzionale.

Car Application – POST (add)

Prima operazione dal vivo: **aggiungere** un'auto.

Flusso in Postman: si fa un **POST** su `/cars` con un body JSON (raw), es.:

brandName: Tesla, **modelName:** Model 3, **regNo:** 241G1,

`carType: electric, year: 2020, km: 1000, price: 40000`

Risposta: **201 Created** (il codice HTTP corretto per una creazione riuscita).

Definizione — Codici di stato 2xx: qualunque codice nella famiglia 200 è un successo. **201 Created** = risorsa creata; **204 No Content** = successo senza corpo nella risposta.

Analisi del codice:

- **@PostMapping** sulla collezione `/cars`, metodo `addCar` (il nome è irrilevante, conta la mapping e l'URI).
- **Definizione — @RequestBody:** dice a Spring di prendere il corpo JSON della richiesta e istanziare l'oggetto `Car` con quei dati. Tutti i dettagli finiscono nell'oggetto.
- **Definizione — Location header e UriComponentsBuilder:** dopo aver creato la risorsa, si costruisce l'URI dove il client potrà ritrovarla (`/cars/{regNo}`) usando `UriComponentsBuilder`, e lo si inserisce nell'**header Location** della risposta. Così il client sa dove recuperare/modificare/eliminare quella risorsa.
- **Definizione — ResponseEntity:** una classe che permette di costruire la risposta HTTP componendola: stato (**201 Created**), header (Location), body (il DTO). È tipizzata `ResponseEntity<CarDTO>`.

Logica nel service (`addCar`):

1. L'auto arriva con `id` nullo (normale, non è ancora nel DB).
2. Si chiama `findByRegNo` per vedere se il numero di targa esiste già. Se `isPresent()` è vero → **lancia un'eccezione** `CarRegNoAlreadyExistsException` (i numeri di targa sono unici, non si può duplicare).
3. Se non esiste, si chiama `save` (metodo gratuito del repository).
4. L'auto salvata ha ora la chiave primaria; la si mappa in DTO (senza id) e si restituisce.

Gestione dell'eccezione: nel `@ControllerAdvice` c'è un handler per `CarRegNoAlreadyExistsException`. Riceve l'eccezione e la `WebRequest`, costruisce una **risposta d'errore dettagliata** (una classe annotata `@Data` di Lombok, che genera getter/setter/toString>equals/hashCode/costruttore), e la restituisce con stato **400 Bad Request** e un messaggio significativo (es. "il numero di targa esiste già nel database").

Test: aggiungendo di nuovo la stessa auto → **400 Bad Request** con messaggio chiaro. Con targhe diverse → più auto aggiunte con successo (2 Tesla + 1 Toyota). Un POST su risorsa individuale → **405 Method Not Allowed**.

Car Application – GET (retrieve)

Operazione di **recupero** con il verbo GET.

Casi mostrati in Postman:

- GET `/cars` → tutte le auto
- GET `/cars/241G1` → una singola auto per targa (**case sensitive**)
- GET `/cars/2x199` (targa inesistente) → **404 Not Found** con messaggio d'errore

- GET /cars/car?brandName=Tesla → tutte le Tesla → **200 OK**

Definizione — Path variable vs Request parameter: due modi diversi di passare dati nell'URL.

- **Path variable** (@PathVariable): fa parte del percorso, es. /cars/241G1 → 241G1 è la variabile di percorso.
- **Request parameter** (@RequestParam): viene dopo il ?, es. /cars/car?brandName=Tesla.

Analisi codice:

- @GetMapping su /cars → getAllCars. Si crea una lista di CarDTO, si chiama findAll del repository, e per ogni auto si mappa in DTO (senza chiave primaria) e si aggiunge alla lista. Il framework serializza la lista in JSON automaticamente.
- @GetMapping("/{carRegNo}") con @PathVariable: **la path variable nella mapping e il parametro del metodo devono corrispondere**. Si chiama il service, che chiama findByRegional. Se Optional è vuoto → lancia CarNotFoundException → gestita globalmente → **404 Not Found**. Altrimenti mappa in DTO e restituisce **200 OK**.
- **Affinare la GET con params:** per il filtro per marca si usa @GetMapping(path="/car", params="brandName"). Significa: un GET su /cars/car?brandName=... arriva a questo metodo. **L'identificatore brandName (con la N maiuscola) deve essere identico** in Postman e nel codice.

Car Application – PUT (update)

Aggiornare un record con PUT.

Definizione — Semantica di PUT: POST serve ad aggiungere a una collezione; PUT serve ad aggiornare una **singola risorsa** esistente. Quindi PUT sulla collezione → **405 Method Not Allowed**.

Esempio: si aggiorna una Tesla da "Model 3" a "Model Y" con PUT /cars/241G2 e il nuovo body. Risposta: **204 No Content** (successo, nessun corpo nella risposta — dice al client "non serve che analizzi il body, ha funzionato").

Due gotcha importanti:

1. Se sbagli URL (es. /cars/ invece di /cars) puoi ricevere **404 Not Found** invece di 405. Attenzione all'URI esatto.
2. **Il numero di targa nell'URI deve corrispondere al numero di targa nel body**. Se non corrispondono → si lancia CarNumbersDoNotMatchException → **400 Bad Request** con un messaggio che mostra sia la targa dell'URI sia quella del body.

Analisi codice — nel repository l'update usa una **query personalizzata**:

- **Definizione — @Query:** annotazione che permette di scrivere la propria query (JPQL/SQL) quando le query derivate non bastano. Tipicamente @Query va bene per le SELECT.

- **Definizione — @Modifying:** per query di INSERT/UPDATE/DELETE serve aggiungere @Modifying accanto a @Query.
- **Definizione — @Transactional:** rende il metodo **atomico** — o viene eseguito completamente, o per niente. Non devi gestire commit/rollback a mano, ci pensa JPA. Fondamentale per le operazioni di scrittura.

L'update sfrutta il numero di targa (unico) per identificare l'auto e aggiornarne tutte le proprietà.

Car Application – DELETE (remove)

Eliminare con DELETE.

Casi in Postman:

- DELETE /cars/2x1G99 (inesistente) → **404 Not Found** con messaggio
- DELETE /cars/241G2 (esistente) → **204 No Content**, record sparito dal DB
- DELETE /cars (sulla collezione) → cancella tutte, **204 No Content**

Analisi codice:

- **Delete all:** @DeleteMapping su /cars → chiama deleteAll del repository (metodo gratuito di JPA, SQL generato in background).
 - **Delete one:** @DeleteMapping("/{carRegNo}") con path variable → chiama deleteCar nel service. Si fa prima findByRegNo: se Optional è vuoto → CarNotFoundException (la stessa usata dalla GET). Se presente, si chiama deleteByRegNo (query derivata: convenzione deleteBy + nome proprietà). Anche qui @Transactional per garantire l'atomicità.
-

Car Application – HEAD

Definizione — verbo HTTP HEAD: fa esattamente la stessa cosa di GET, **ma senza il corpo** nella risposta. Serve a dimostrare al client che una risorsa esiste, senza trasferire i dati. È supportato **automaticamente** dal framework — non serve scrivere codice.

Esempi:

- HEAD /cars → **200 OK**, body vuoto → la risorsa esiste
 - HEAD /cars/{regNo} su targa esistente → **200 OK**, body vuoto
 - HEAD su targa inesistente → **404 Not Found**, body vuoto (mentre la GET equivalente restituirebbe il body con i dettagli dell'errore)
-

Car Application – OPTIONS

Definizione — verbo HTTP OPTIONS: dice al client **quali verbi HTTP sono supportati** da una risorsa, tramite l'header **Allow** nella risposta.

Di default Spring restituisce un set di verbi, ma non sempre corretto per la semantica REST (es. suggerisce PUT sulla collezione, o POST sulla singola risorsa). Il docente **personalizza** OPTIONS.

Dettaglio tecnico: **non esistono** le annotazioni `@OptionsMapping` e `@HeadMapping` come scorciatoie. Bisogna usare `@RequestMapping(method = RequestMethod.OPTIONS)`.

Configurazione finale corretta:

- `/cars` (collezione) → Allow: HEAD, GET, **POST**, DELETE
- `/cars/{regNo}` (singola) → Allow: HEAD, GET, **PUT**, DELETE
- `/cars/car` (quello dei request param) → Allow: HEAD, GET (sola lettura)

Il contrasto chiave: la **collezione permette POST**, la **singola risorsa permette PUT**. Si costruisce con `ResponseEntity` impostando lo stato 200 e l'header `Allow` con la lista (varargs) dei metodi permessi.

Query by Car Type

Aggiungere una **nuova query** (per tipo di auto, es. tutte le elettriche), stavolta mostrando il processo dal vivo. Il pattern per aggiungere una query attraversa tutti i livelli:

1. **Repository:** aggiungi `findAllByCarType( . . . )` (query derivata; deve esistere la proprietà `carType` nell'entità).
2. **Service (interfaccia):** aggiungi il metodo `getAllCarsByCarType`.
3. **Service (implementazione):** implementa il metodo — crea lista DTO, chiama il repository, mappa ogni auto in DTO.
4. **Controller:** aggiungi `@GetMapping(path="/car", params="carType")` con `@RequestParam carType`.

Test: con 2 auto electric e 1 "ICE" (Internal Combustion Engine = benzina/diesel), ?
`carType=electric` restituisce le 2 Tesla, ?`carType=ICE` restituisce la Toyota. Dimostra quanto è facile far generare l'SQL a JPA e collegarlo al controller.

Gotchas (insidie da evitare)

Un riepilogo di trappole ricorrenti, molto utile:

1. **Request param: gli identificatori devono corrispondere.** Se nel codice usi `@RequestParam("brandName")` (con N maiuscola), in Postman devi scrivere esattamente `brandName`. Lo stesso vale per le **path variable**: l'identificatore nella mapping e nel parametro del metodo devono combaciare.
2. **year è parola riservata in SQL.** Diventa blu nell'editor. Genera un messaggio SQL oscuro. Soluzione: usare `@Column(name = "\"year\"")` o rinominare.
3. **Gli identificatori delle proprietà nel JSON devono corrispondere alle proprietà del bean.** Se la proprietà si chiama `brand` e nel POST mandi `brandName` con la n minuscola sbagliata, ottieni "Null not allowed for column". Spring cerca il setter/costruttore corrispondente e, se non trova per via del case, il campo resta null.

4. **Relazione diretta nome-metodo-repository** ↔ **proprietà-entità**. `findByRegNo` implica una proprietà `regNo` nell'entità. Se non c'è, errore all'avvio: "No property regNo found for type Car". (Ottimo che l'errore sia esplicito.)
 5. **Attenzione al trailing slash nell'URI**. `/cars` funziona; `/cars/` dà 404 Not Found (se il tuo codice non lo prevede).
 6. **Come aggiungere una query, in sintesi**: aggiungi il metodo derivato nel repository (basato sul nome della proprietà del bean), poi il service, poi il controller. Non scrivi né SQL né JPQL: `findByX` e `findAllByY` li fa JPA per te. Queste sono le **derived queries**.
-

Adding HATEOAS (Hypermedia As The Engine Of Application State)

Concetto più avanzato che porta l'app a un livello REST più "puro".

Definizione — HATEOAS: *Hypermedia As The Engine Of Application State*. L'idea è imitare l'esperienza di navigazione umana su un sito. Analogia con Amazon: entri dalla home, ricevi dei **link**, clicchi e passi a un altro *stato* dell'applicazione. In HATEOAS **è il server a guidare l'interazione**: risponde includendo dei link nella rappresentazione, e il client può seguire quei link per passare allo stato successivo. Da qui il nome REST = *Representational State Transfer* (trasferimento di stato rappresentazionale).

Passi per aggiungerlo:

1. **Dipendenza**: da `start.spring.io` si cerca "HATEOAS", si copia il pezzo di XML nel `pom.xml`, e si **clicca la piccola icona fluttuante** di Maven per ricaricare la build (facile da mancare).
2. **Il DTO estende `RepresentationModel`**: il `CarDTO` deve fare `extends RepresentationModel<CarDTO>` (dal package `org.springframework.hateoas`). Serve perché eredita il metodo `add( . . . )` per aggiungere link.
3. **Nel controller** si importano due metodi statici da `WebMvcLinkBuilder`: `linkTo` e `methodOn`.

Definizione — `rel` e `href`: seguendo il protocollo dei link, ogni link ha due attributi: `rel` (la *relazione* della risorsa, cioè "a cosa serve questo link") e `href` (il link vero e proprio).

Meccanismo (per ogni auto restituita nel GET all):

- `add( . . . )` aggiunge un link.
- `linkTo(methodOn(CarController.class).qualcheMetodo( . . . ))` fa un'**invocazione fittizia (dummy)** sul metodo target del controller per ricavarne l'URL. Essendo fittizia, si passa `null` dove opportuno.
- `.withSelfRel()` crea il link **self** (come sei arrivato a questa rappresentazione: un GET su questo URL).
- `.withRel("addCar")`, `.withRel("getCar")`, `.withRel("updateCar")`, `.withRel("deleteCar")` creano i link con relazioni personalizzate verso gli altri metodi (`addCar`, `getCarDetails`, `updateCar`, `deleteCarDetails`).

Nota importante: i nomi dei metodi passati a `methodName` sono **espliciti** — devono essere i nomi reali dei metodi nel controller, non stringhe arbitrarie.

Risultato in Postman: ogni auto restituita ha ora una sezione di link (self, e come aggiungere/recuperare/aggiornare/eliminare), con href che puntano alle risorse giuste (targhe diverse per auto diverse). Il server "guida" il client.

Assignment Solution (applicazione Books)

Da qui il corso risolve l'**assignment**: costruire la stessa app ma per **libri**. È tutto parallelo alla parte Cars, quindi riassumo evidenziando ciò che cambia.

Configuration: stesso setup su start.spring.io. Group `com.library`, artifact `books`. Stesse dipendenze (Spring Web, H2, Spring Data JPA, Lombok). Si importano in IntelliJ i file `application.yml` e `schema.sql` forniti nello zip (insieme al PDF dell'assignment). L'entità libro ha: `id` (chiave primaria generata), `book_title`, autore, editore, `isbn`, anno di pubblicazione, prezzo. **L'ISBN è unico per ogni libro**, quindi fa da chiave "di business" (come la targa per le auto). Si crea un `BookController` di test con una `@GetMapping` per verificare che tutto parta.

Packages Overview and Entity Bean: stessa struttura a livelli (controller, service, repository, entity, exception). L'entità `Book` è `@Entity`, con `@Id + @GeneratedValue(IDENTITY)`, e `@Column` solo dove il nome proprietà ≠ nome colonna (es. `bookTitle` → `book_title`, `yearPublished` → `year_published`). Il `BookRepository` extends `JpaRepository<Book, Long>`. C'è il `BookDTO` (senza chiave primaria), il `BookMapper`, l'interfaccia `IBookService` con l'implementazione annotata `@Service`, il `BookController` (`@RestController`, `@RequestMapping("/books")`) con DI del service, e il `@ControllerAdvice` globale.

Adding books (POST): `POST /books` con body JSON (es. "Learning Java with Projects", autori Sean Kennedy e Mike van Putten, ISBN, anno 2023, prezzo 39). → **201 Created**, con header `Location /books/{isbn}`. Logica identica alle auto: `findByIsbn` → se presente lancia `BookIsbnAlreadyExistsException` → **400 Bad Request**; altrimenti `save` e restituisci il DTO. Riaggiungere lo stesso libro → 400.

Retrieving books (GET): `GET /books` (tutti → 200), `GET /books/{isbn}` (uno; se ISBN inesistente → `BookIsbnNotFoundException` → **404**). La path variable `{isbn}` deve corrispondere al `@PathVariable isbn`.

Deleting books (DELETE): `DELETE /books/{isbn}` (uno → 204; inesistente → 404), `DELETE /books` (tutti → 204). Usa `deleteByIsbn` (derivata, con `@Transactional`) e `deleteAll` (gratuito).

Updating a book (PUT): `PUT /books/{isbn}` con body per aggiornare (es. abbreviare "Michael van Putten" in "MVP") → **204 No Content**. Se l'ISBN dell'URI ≠ ISBN del body → `BookIsbnMismatchException` → **400**. L'update nel repository usa una **query**

personalizzata con `@Query` (JPQL: `update Book b set b.bookTitle = ?1 ...` con parametri posizionali `?1, ?2, ...`) + `@Modifying` + `@Transactional`.

Custom query using query/request parameters: questa lezione ha un insegnamento importante. Obiettivo: trovare tutti i libri di un autore, es. `GET /books/book?author=Kennedy`.

- Prima si prova una **query derivata** `findByAuthor`. Ma l'entità ha la proprietà `authors` (plurale) → errore all'avvio: "No property author found... Did you mean authors?". Si rinomina in `findByAuthors`.
- Problema: **le query derivate usano una corrispondenza esatta**. Va bene per l'ISBN, ma per l'autore no: se il DB ha "Sean Kennedy and Michael van Putten", cercare "Kennedy" restituisce **204 No Content** (nessun match); e invertendo l'ordine dei nomi non trova nulla.
- Soluzione: **query personalizzata** con `@Query` e l'operatore SQL **LIKE** (che agisce come carattere jolly): `select b from Book b where b.authors like %:authorName%`. L'identificatore `:authorName` (parametro named) deve corrispondere al `@Param("authorName")` nel metodo.
- Ora `?author=Kennedy` trova entrambi i libri **indipendentemente dall'ordine dei nomi**. Questa è la potenza di `@Query` + **LIKE**.

405 Method Not Allowed: si forzano i due casi: POST su risorsa individuale (`POST /books/{isbn}`) e PUT sulla collezione (`PUT /books`). Entrambi restituiscono **405**. Nel codice si mappano quei metodi e si restituisce semplicemente `ResponseEntity` con stato `METHOD_NOT_ALLOWED`. Regola: POST solo sulla collezione (creare), PUT solo sulla singola (aggiornare).

Supporting OPTIONS: come per le auto. Con `@RequestMapping(method = OPTIONS)` si personalizza l'header `Allow`:

- `/books` (collezione) → HEAD, GET, **POST**, DELETE
- `/books/{isbn}` (singola) → HEAD, GET, **PUT**, DELETE
- `/books/book` (request param) → HEAD, GET (sola lettura)

Concetti trasversali da portarsi a casa

Se dovessi condensare tutto il corso in pochi principi ricorrenti:

- **L'architettura a tre livelli** con responsabilità separate: il controller riceve e risponde, il service valida e applica logica, il repository parla col DB. Ogni livello parla solo con quello sotto.
- **IoC e Dependency Injection:** tu non fai `new`; annoti le classi (`@RestController`, `@Service`, `@Component`), le dichiari come dipendenze, e Spring le inietta. Lombok (`@AllArgsConstructor`, `@Data`) genera costruttori e accessor.
- **JPA fa il lavoro pesante:** `JpaRepository` dà metodi gratis; le **query derivate** si generano dal nome del metodo (che deve rispecchiare le proprietà dell'entità); `@Query` + `@Modifying` + `@Transactional` per le query personalizzate e le scritture atomiche.

- **REST fatto bene:** verbi giusti sulle risorse giuste (POST su collezione, PUT su singola), codici di stato corretti (201, 204, 400, 404, 405), path variable vs request param, HEAD e OPTIONS supportati, header Location e Allow, e HATEOAS per link guidati dal server.
- **Il DTO per la sicurezza:** non esporre mai la chiave primaria al client; usa un mapper per restituire tutto tranne l'id.
- **Gestione centralizzata degli errori:** un unico `@ControllerAdvice` che intercetta tutte le eccezioni personalizzate e costruisce risposte d'errore dettagliate e significative.